

Object Oriented Programming



Subject Code: MCA 4121

Course Outcomes:

2

At the end of the program the student must be able to:

- ❖ Describe and differentiate the procedural and object oriented paradigm with concepts of data types, streams, classes, functions, data and objects. Understand and implement the OOP idea of data hiding, abstraction, encapsulation and polymorphism, reference arguments, static members, pointers.
- ❖ Identifying and utilizing the concepts of constructors, destructors, function overloading, operator overloading and dynamic memory management operators.
- ❖ Classify inheritance with the understanding of early and late binding, usage of virtual base classes, virtual functions.
- ❖ Explain and use the concept of files, exception handling, generic programming.
- ❖ To demonstrate usage of three entities of STL: containers, algorithms and iterators.

Text Books:

1. Robert Lafore, "Object Oriented Programming in C++", 4th Edition, Galgotia Publications Pvt.

Reference Books:

1. *Deitel & Deitel*, "C++ - How to program", Prentice Hall.
2. K R Venugopal, Rajkumar, T Ravishankar, "Mastering C++", Tata McGrawHill, 2005.
3. Herbert Schildt, "The Complete Reference- C++", Tata McGraw Hill, 2000.
4. Stanley B Lippman, Josee Lajoie and Barbara E Moo "C++ Primer", 5th Edition, 2012.

Websites

- A C++ online tutorial:
<http://www.cplusplus.com/doc/tutorial/>
- The C++ FAQ:
<http://www.parashift.com/c++-faq-lite>
- The homepage of Bjarne Stroustrup, the inventor of C++:
<https://www.stroustrup.com/C++.html>

And many, many more!

Mode of Evaluation – Theory

➤ Internal Assessment

- ✓ Mid-Term Exam : 30 marks
- ✓ 3 assignments : 20 marks
- ✓ **Total Internal semester marks: 30 + 20 = 50**

• End Semester Assessment

- ✓ 3 hour paper with weightage of 50 marks
- **Total Assessment = 50 (internal)+50(end semester) = 100 marks**

C++ and C

- C is a subset of C++.

Advantages: Existing C libraries can be used, efficient code can be generated.

- ✓ is a better C
- ✓ supports data abstraction
- ✓ supports object-oriented programming
- ✓ supports generic programming



C++ and Java

- Java is a full object oriented language, all code has to go into classes.
- C++ - in contrast - is a hybrid language, capable both of functional and object oriented programming.



So, C++ is more powerful but also more difficult to handle than Java.

Compiling C++ programs on Linux

- Use

```
#include <iostream>  
using namespace std;
```

- Use .cpp as extension eg: prg1.cpp
- Use the following compile command

```
g++ prg1.cpp
```

To run the program: **./a.out**

Other Editors

- Turbo C++
- Borland C++
- DevC++
 - <http://www.bloodshed.net/dev/devcpp.html>
- Sublime
- Code blocks
- Microsoft Visual C++

Program Hello

```
// This program outputs the message Hello! to the screen
#include <iostream>
using namespace std;

int main()
{
    cout <<"Hello!"<< endl;
    return 0;
}
```

Namespaces

- Namespaces are used to prevent name conflicts
- Namespace **std** is used routinely to cover the standard C++ definitions, declarations, and so on for standard C++ library
- The **using** directive is equivalent to using a declaration for each item in a namespace

Keyboard & Screen I/O in C++

- It is perfectly valid to use the same I/O statements in C++ as in C -- The very same *printf*, *scanf*, and other *stdio.h* functions that have been used until now.
- However, C++ provides an alternative with the new stream input/output features.

```
#include <iostream>  // No ".h" on std headers
```

```
using namespace std; // To avoid things like
```

```
// std::cout and std::cin
```

```
cin // Used for keyboard input (std::cin)
```

```
cout      // Used for screen output (std::cout)
```

Keyboard & Screen I/O in C++

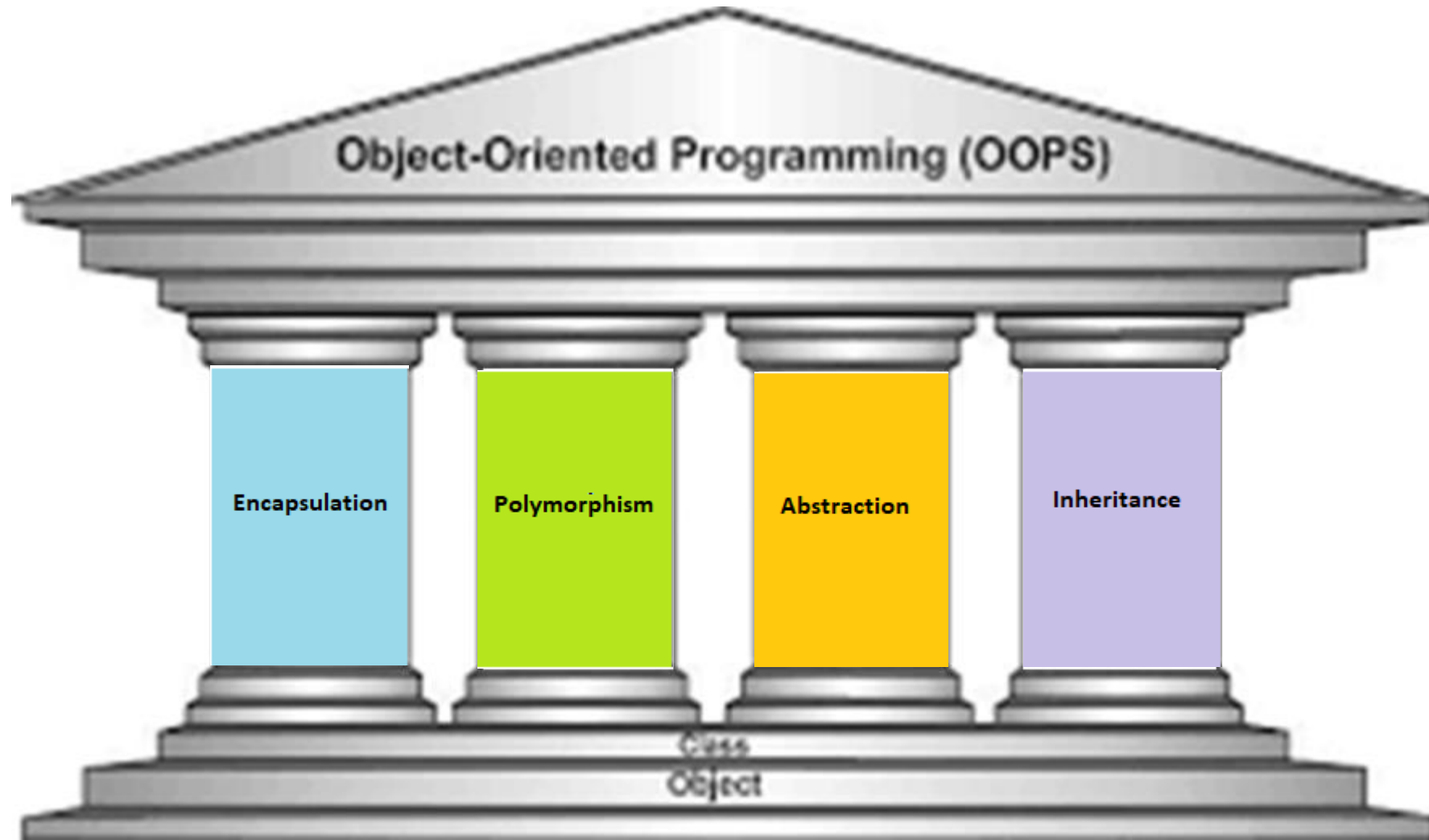
- Since **cin** and **cout** are C++ objects, they are somewhat "intelligent":
 - They **do not** require the usual format strings and conversion specifications.
 - They **do** automatically know what data types are involved.
 - They **do not** need the address operator, &.
 - They **do** require the use of the stream extraction (>>) and insertion (<<) operators.
- The next slide shows an example of the use of **cin** and **cout**.

Classes

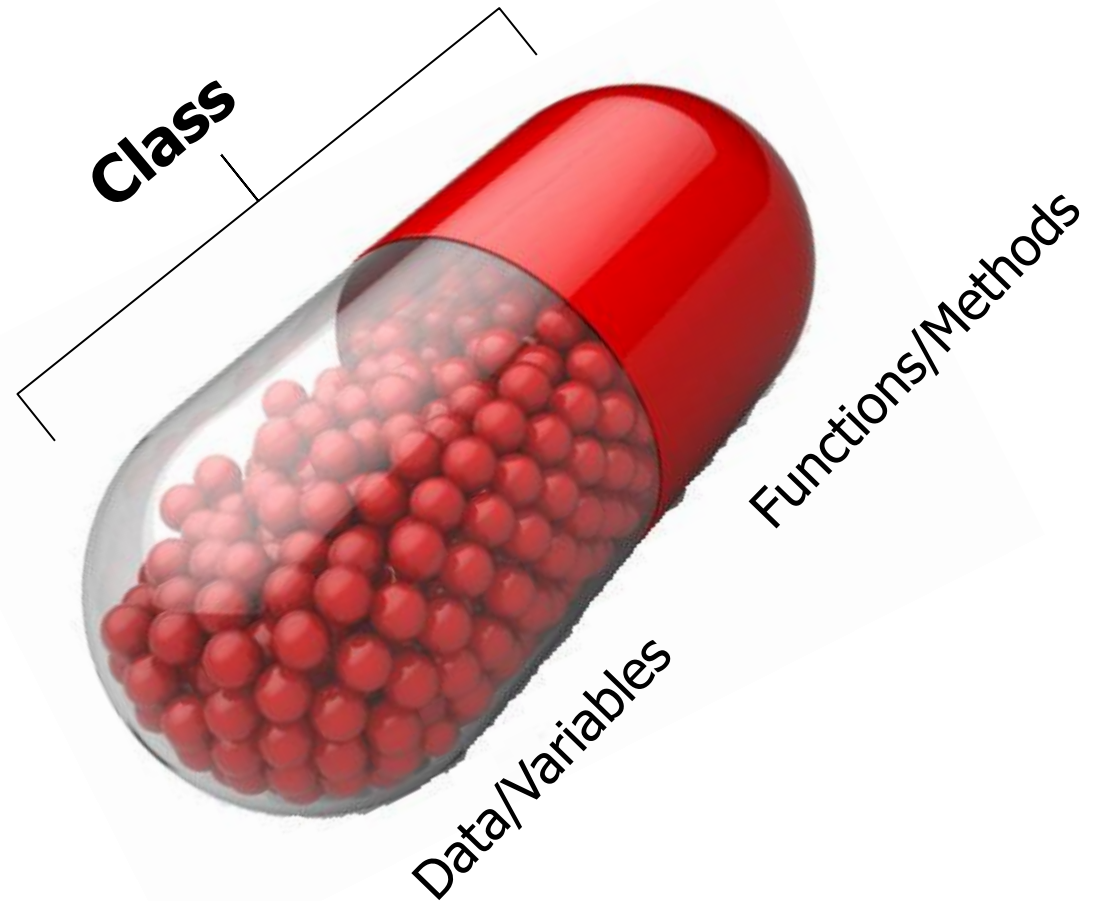


- ❑ **The entire set of data and code of an object can be made a user defined data type with the help of a class.**
- ❑ **Objects are variables of the type class.**
- ❑ **Once a class has been defined, we can create any number of objects belonging to that class.**
- ❑ **Classes are user defined data types and behave like the built-in types of a programming language.**
- ❑ **Internal data of a class – member data**
- ❑ **Internal functions of a class – member functions**

Object Oriented Programming Concepts

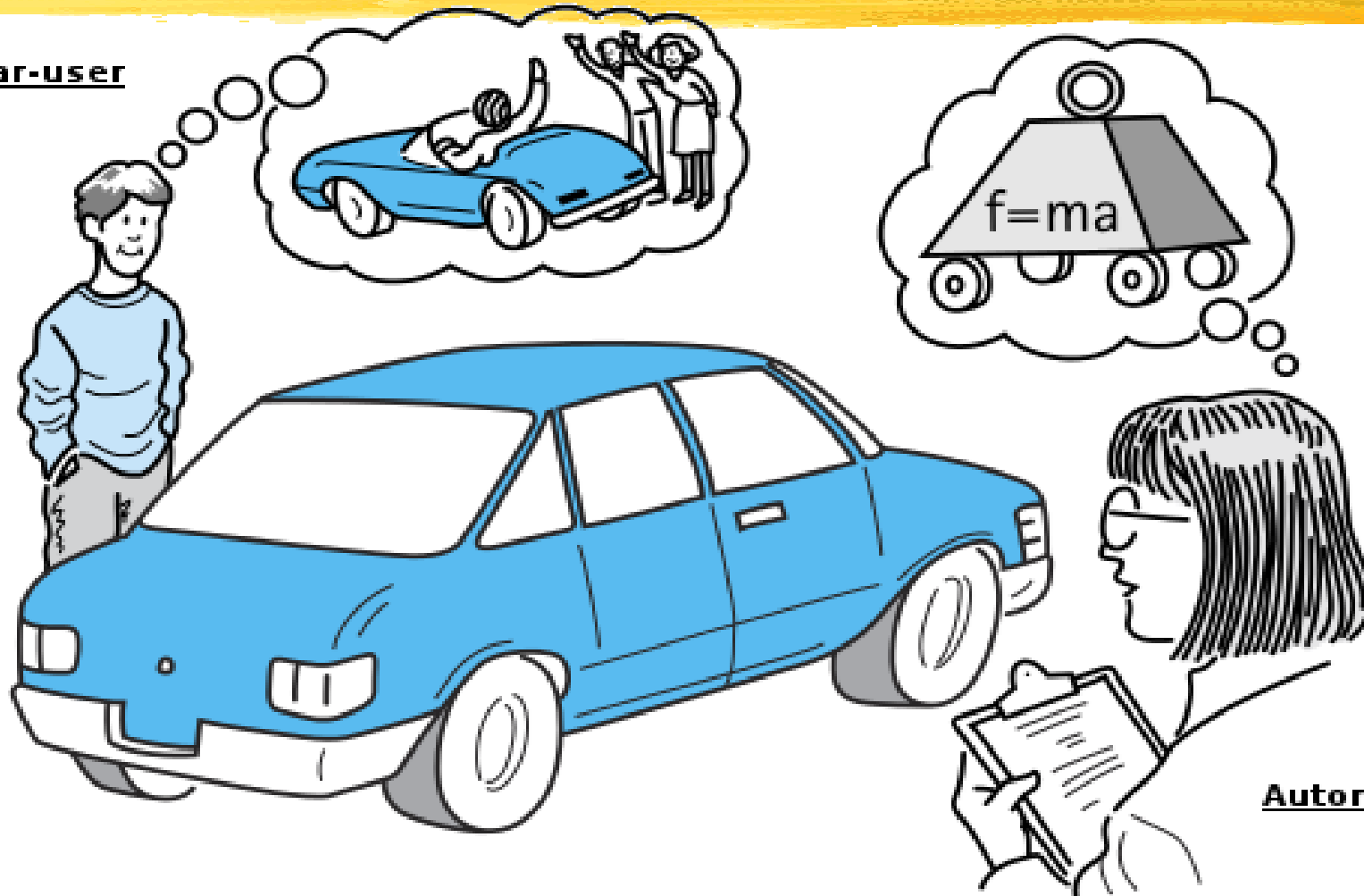


Encapsulation



Abstraction

Any car-user

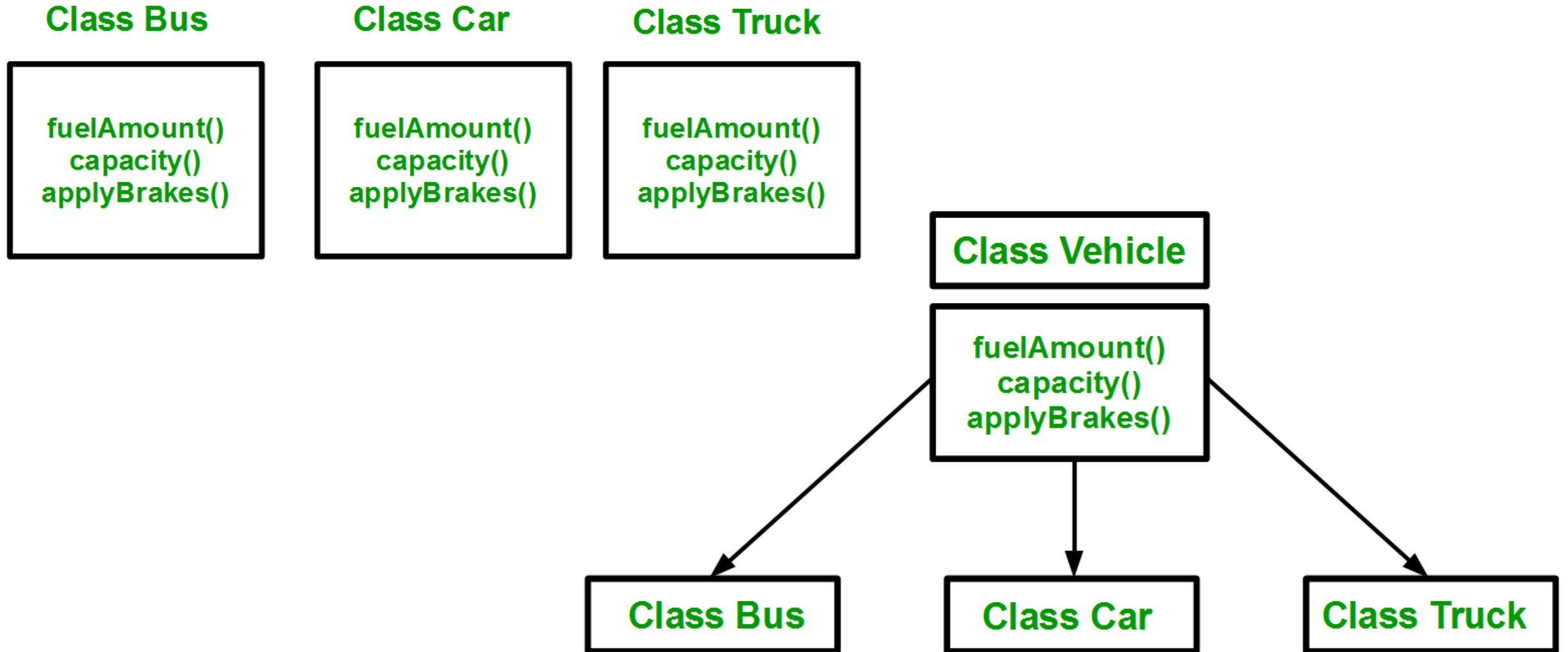


Automobile Engg.

An abstraction includes the essential details relative to the perspective of the viewer

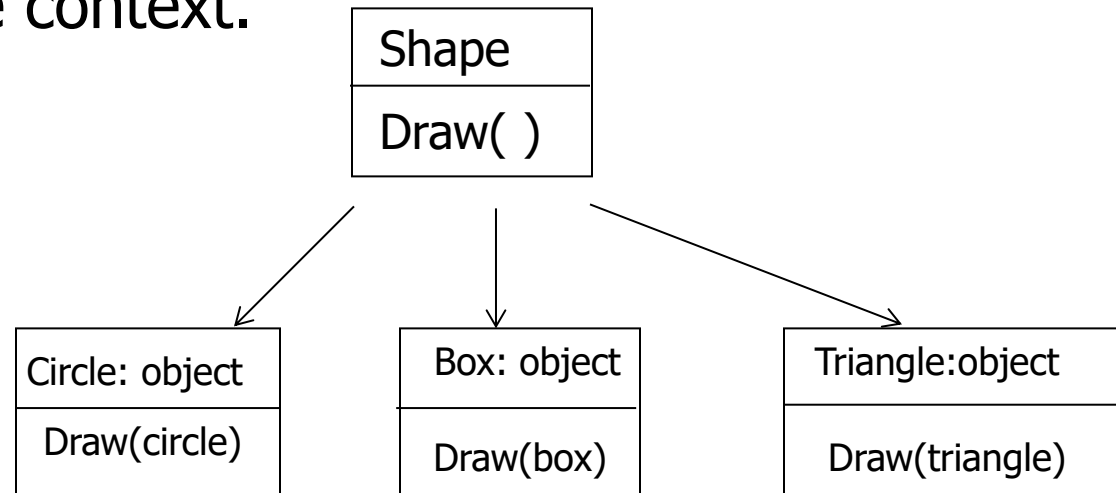
Inheritance

□ Process by which one object acquires the properties of another object



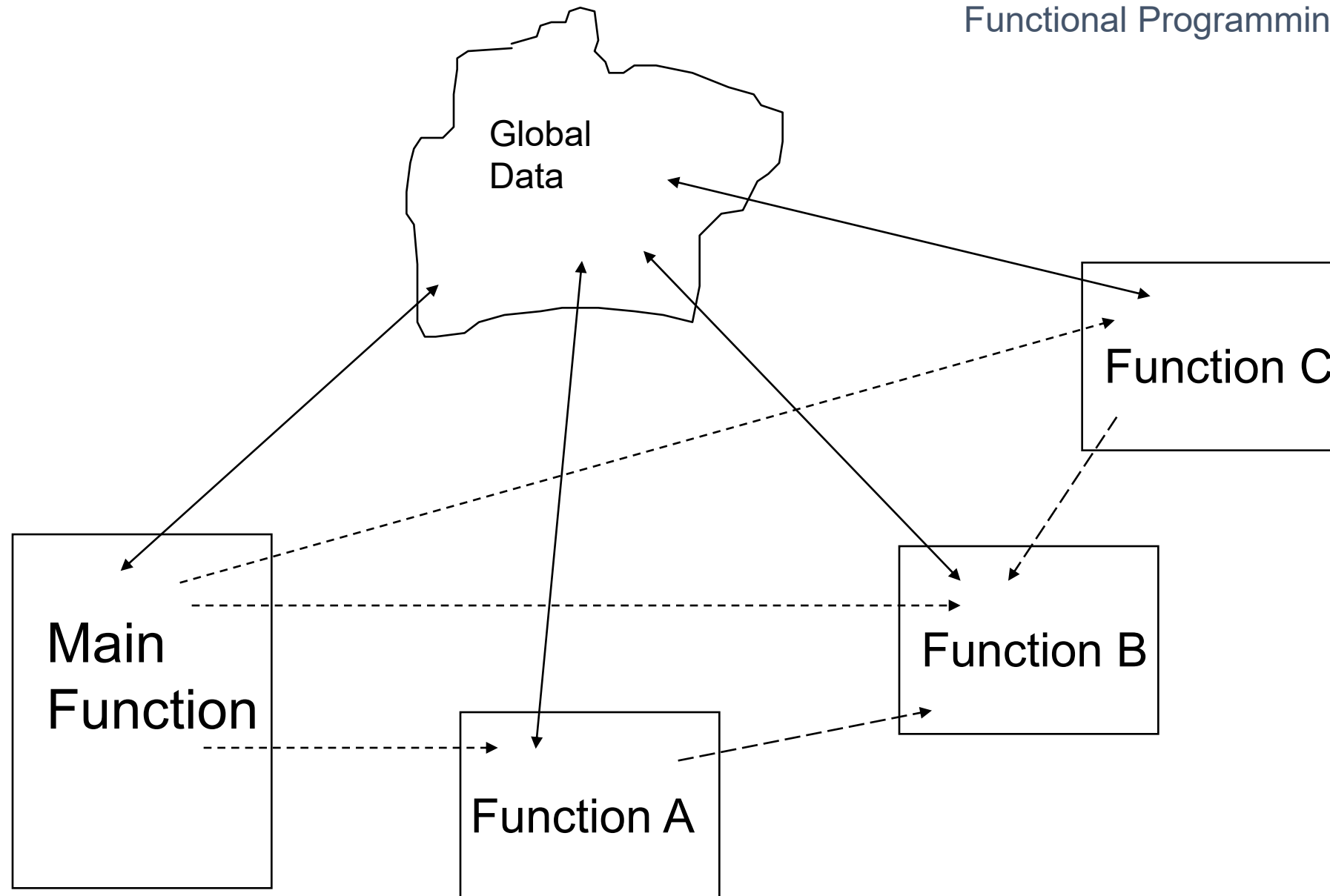
Polymorphism

- Polymorphism means the ability to take more than one form. The behavior depends upon the types of data used in the operation.
- This is something similar to a particular word having several different meanings depending on the context.



Procedure Oriented Programming

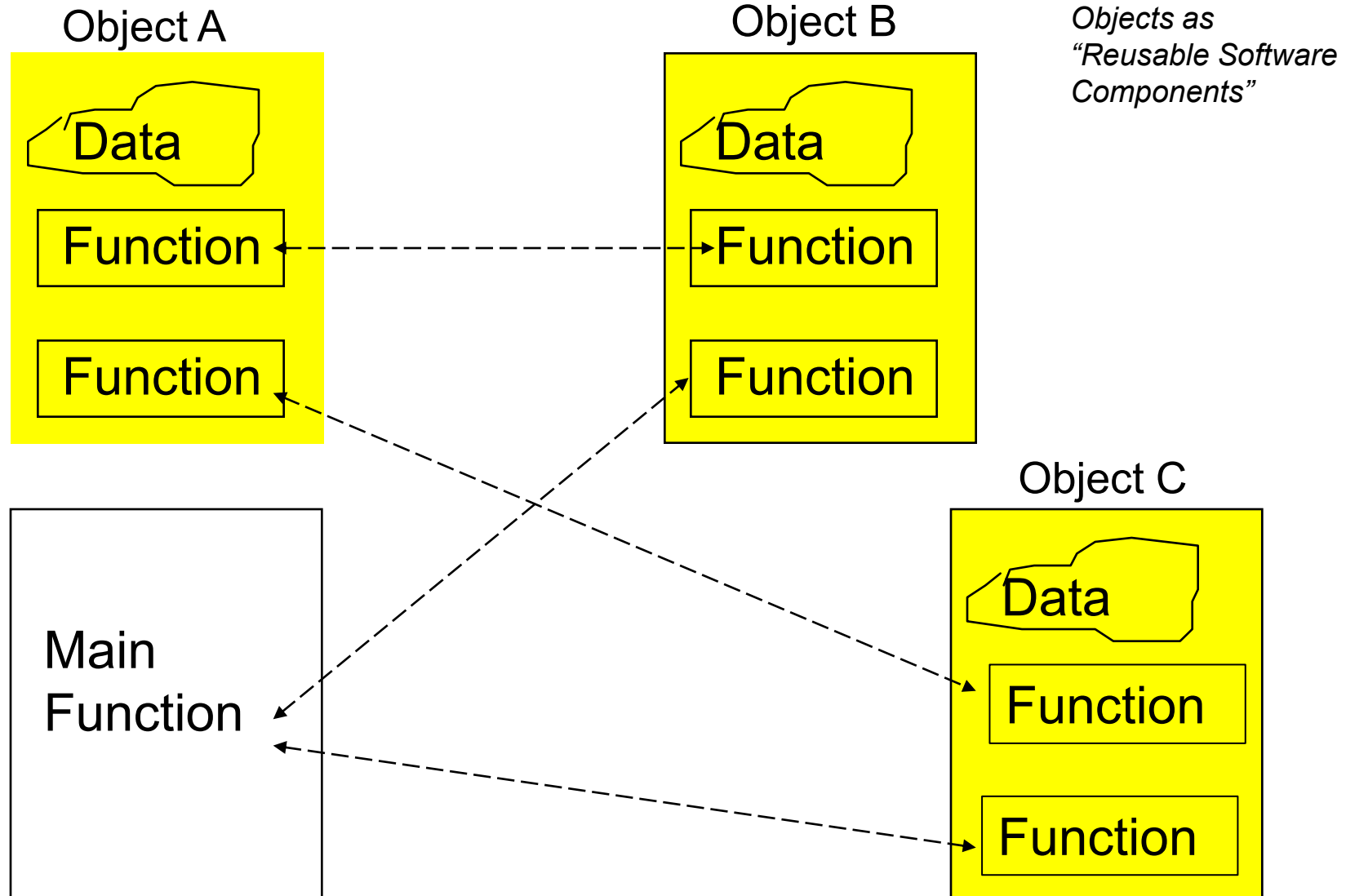
- Emphasis is on doing things (algorithms/ procedures).
- Large programs are divided into smaller programs known as functions.
- Most of the functions share global data.
- Data moves openly around the system from function to function.
- Uses top-down approach in program design.



Object Oriented Programming

- ✓ Emphasis is on data rather than procedure.
- ✓ Programs are divided into what are known as objects.
- ✓ Data is hidden and cannot be accessed by external functions.
- ✓ Functions that operate on the data of an object are tied together in the data structure.
- ✓ Objects may communicate with each other through functions.
- ✓ Follows bottom-up approach in program design

Object-Oriented Programming



Concepts of Object Oriented Programming

- Objects
- Classes
- Data abstraction and encapsulation
- Inheritance
- Polymorphism
- Dynamic binding
- Message passing

Reminder Pop Quiz!

- Objects have:
 - A. Both attributes and behaviour
 - B. Only attributes
 - C. Only behaviour
 - D. Attributes often, behaviour less often

Benefits of Object Oriented Programming

- Through inheritance, we can eliminate redundant code and extend the use of existing classes.
- The principle of data hiding helps the programmer to build secure programs that cannot be invaded by code in other parts of the program.
- It is possible to map objects in the problem domain to those in the program.
- It is easy to partition the work in a project based on objects.
- Object oriented systems can be easily upgraded from small to large systems.
- Software complexity can be easily managed

Applications written in C++

- Apple OS and iPod user interface
- Google file system, Google chromium
- Mozilla Firefox & email client thunderbird
- Symbian OS
- Win 95, 98, XP, IE, Visual Studio, SQL
- WinAmp media Player
- Adobe systems: Photoshop, Illustrator
- Alias system: Maya
- Amadeus: in excess of 5000 transactions per second, 200000 terminals connected, 24/7 operations. All Unix based-server apps completely in C++

<http://www.stroustrup.com/applications.html>